Claude Code — AI-Augmented Software Engineering

WEEKLY ASSIGNMENT — WEEK 2

The Spec-Driven Feature Factory

Duration: 4–5 hours | **Format:** Solo | **Sessions Covered:** Session 3 (Advanced Prompt Engineering) + Session 4 (Spec-Driven Development)

Deliverables: GitHub Repository + Written Report (answers to all questions below)

Assignment Brief

Your product manager just dropped a feature request on your desk. Instead of diving into code, you will **build it the spec-driven way**: formal specification first, then AI-generated code, then auto-generated tests, with full traceability from requirement to implementation.

You will create a **YAML prompt library**, write a **structured specification**, generate code from the spec, auto-generate tests, and validate everything against the original requirements.

■

The Feature Request

Build a **URL Shortener Service** with the following requirements (from your PM):

- **Core:** Shorten long URLs into unique short codes (e.g., abc123)
- **Redirect:** Visiting the short URL redirects to the original
- **Analytics:** Track click count, last accessed timestamp, referrer
- **Expiry:** URLs can have optional expiration dates
- **Validation:** Reject invalid URLs, duplicates, and malicious links
- **API:** RESTful API with proper error responses

You may use **any language or framework**. The deliverable is the process and artifacts, not a specific tech stack.

Starter scaffolds (pick one to bootstrap your project):

Stack	Scaffold Repository	What You Get
Node + Express	hagopj13/node-express-boilerplate	Express + MongoDB + JWT + Docker

Node + Fastify	fastify/fastify-starter	Fastify + TypeScript + Swagger
Python + FastAPI	tiangolo/full-stack-fastapi-template	FastAPI + SQLAlchemy + pytest + Docker
Python + Flask	miguelgrinberg/flasky	Flask + SQLAlchemy + blueprints
Go + Chi	go-chi/chi (examples/)	Chi router + middleware + REST
Java + Spring	spring-projects/spring-petclinic	Spring Boot + JPA + tests
TypeScript	honojs/starter	Hono + edge runtime + testing
Rust	tokio-rs/axum (examples/)	Axum + Tower + async patterns

Reference implementations (study these for ideas, do NOT copy code — the point is to generate from spec):

- **github.com/YOURLS/YOURLS** — PHP URL shortener (mature, full-featured, good for studying the domain)
- **github.com/thedevs-network/kutt** — Node.js URL shortener (modern, TypeScript, good API design)
- **github.com/chhpt/tiny-url** — Minimal Go URL shortener (clean architecture, good patterns)

Part 1: YAML Prompt Library (45 min)

Before writing any code, build a reusable prompt library in YAML format.

1.1 Create 4 YAML Prompt Templates

- **prompts/spec-writer.yaml** — Role: product analyst. Converts feature requests into structured specs with SHALL/MUST language, Gherkin scenarios, and acceptance criteria
- **prompts/architect.yaml** — Role: senior architect. Generates implementation plans with component breakdown, interface definitions, and data models
- **prompts/code-reviewer.yaml** — Role: security reviewer. Reviews code for OWASP vulnerabilities, returns JSON with severity scores
- **prompts/test-generator.yaml** — Role: QA engineer. Generates comprehensive tests from specs including edge cases

Each YAML template must include: name, version, role, task, output_schema, and tags.

Use {{ variable }} template syntax for parameterization.

Part 2: Structured Specification (45 min)

2.1 Generate the Spec

- Use your spec-writer.yaml prompt to convert the PM's feature request into a formal spec
- The spec must use SHALL/MUST normative language
- Include at least 6 Gherkin scenarios (happy path + error cases + edge cases)

- Include an OpenAPI-style API contract for all endpoints
- Include non-functional requirements (performance, security, rate limiting)
- Save as specs/url-shortener.yaml

2.2 Visual Spec

- Generate a Mermaid sequence diagram for the URL shortening flow
- Generate a Mermaid ER diagram for the data model
- Generate a Mermaid state diagram for URL lifecycle (active → expired → deleted)
- Embed all diagrams in the spec or save in specs/diagrams/

Part 3: Spec-Driven Implementation (90 min)

3.1 Generate Implementation Plan

- Use your architect.yaml prompt to generate a technical plan from the spec
- Plan should reference specific requirement IDs from the spec
- Break into numbered tasks with acceptance criteria

3.2 Implement with Traceability

- Implement the feature task-by-task using Claude Code
- Each file must include a comment linking to the spec requirement: // REQ-SHORT-001
- Use the self-critique loop: Generate → Review (using code-reviewer.yaml) → Fix → Validate

3.3 Schema-Enforced Output

- At least one Claude interaction must use --json-schema flag or inline JSON schema
- Document the schema you used and the validated output

Part 4: Test Generation & Validation (45 min)

- Use your test-generator.yaml prompt to auto-generate tests from the spec
- Tests must cover all Gherkin scenarios from Part 2
- Run the tests and document pass/fail results
- Generate a **traceability matrix**: requirement ID → code file → test file → pass/fail
- If any test fails, fix the code and re-run (document the iteration)

Thinking Questions

Answer these in your written report.

Q1. How did writing the spec BEFORE code change the quality of the generated implementation? Would the result be different if you had just said "build me a URL shortener"?

(Compare spec-driven output to what ad-hoc prompting would produce)

Q2. What was the value of using YAML prompt templates vs. typing prompts ad-hoc? Would you use this approach on your team?

(Consider: reusability, version control, consistency, onboarding)

Q3. Describe your self-critique loop in action. Did Claude find real issues in its own code? What types of issues did it miss?

(Be specific about what the critique caught and what it didn't)

Q4. How complete was your traceability matrix? Were there requirements without tests? Tests without requirements? What does this tell you?

(Gaps in traceability = gaps in coverage = bugs in production)

Q5. What role did visual specs (Mermaid diagrams) play in your process? Did generating them reveal requirements you had missed?

(Diagrams often expose edge cases that text specs hide)

Q6. If your PM changed a requirement mid-sprint (e.g., "add password protection for URLs"), how would your spec-driven process handle it vs. ad-hoc coding?

(Think about delta specs, impact analysis, and test regeneration)

Tactical Questions

Q7. Show your best YAML prompt template. Explain each field (name, version, role, task, output_schema, tags) and why you structured it that way.

(Include the full YAML content)

Q8. Show the JSON schema you used for enforcing structured output. What did the validated output look like?

(Include both the schema and the actual Claude response)

Q9. Show your traceability matrix (requirement → code → test → status). How many requirements had full coverage?

(A table or structured list is fine)

Q10. What percentage of auto-generated tests passed on the first run? What types of failures occurred?

(Be honest — first-run pass rate is a key SDD metric)

Q11. Show a Gherkin scenario and the test code Claude generated from it. How faithful was the implementation to the spec?

(Include both the Given/When/Then and the actual test code)

Q12. What was the total time breakdown across the 4 parts? Which part took longest and why?

(This helps you estimate SDD adoption cost for your team)

Deliverables Checklist

Submit a GitHub repository containing:

- prompts/ — 4 YAML prompt templates (spec-writer, architect, code-reviewer, test-generator)
- specs/url-shortener.yaml — full structured specification with SHALL/MUST, Gherkin, API contract
- specs/diagrams/ — Mermaid diagrams (sequence, ER, state)
- src/ (or equivalent) — implementation with requirement ID comments in code
- tests/ — auto-generated test suite
- docs/traceability-matrix.md — requirement → code → test mapping
- docs/self-critique-log.md — log of generate → review → fix cycles
- REPORT.md — written answers to all 12 questions above